Study Core- Intelligent Study Engine

CCP REPORT

MEMBERS

Eman Siddiqui CT-072

Hafsa Nasir CT-068

TEACHER

Miss Murk Marvi

DISCIPLINE

BCIT (1st Year)

Task A : Application Overview

➢ INTRODUCTION

StudyCore is a console-based application designed to help students plan their exam preparation effectively. Many students struggle with creating study schedules, especially when managing multiple subjects with limited time. This application automates the timetable creation process, making study planning simple, fast, and efficient.

➢ WHAT THE APPLICATION DOES

StudyCore acts as a smart study assistant that generates a personalized timetable based on user input. The user enters details such as subject name, total chapters, days remaining before the exam, and their weakness level. The system then automatically distributes the chapters across available days and reserves the final day for revision. It also provides helpful suggestions to improve study efficiency.

➢ TARGET USERS

The primary users of StudyCore are students preparing for exams. It is especially useful for:

- Students managing multiple subjects
- Students with limited preparation time
- Students who struggle with time management
- Students who want to focus more on weak subjects

Overall, the application is designed to support any student who wants a structured and organized study plan.

➢ CORE FEATURES AND MODULES

❖ *User Profile Module*

Allows users to create and view their profile by entering their name and ID, providing a personalized experience.

❖ *Subject Management Module*

Enables users to add, edit, delete, and view subjects. Each subject stores details like total chapters, days until exam, and weakness level.

❖ *Timetable Generation Module*

This is the main feature of the system. It automatically generates a study plan by:

- Sorting subjects based on exam urgency
- Dividing chapters across available days
- Keeping the last day for revision

❖ *Study Progress Module*

Allows users to track completed chapters and monitor remaining work, helping them stay consistent.

❖ *Warning and Suggestion System*

Provides alerts for heavy workloads and highlights weak subjects, guiding users to focus where needed.

➢ REASON FOR CHOOSING THIS APPLICATION

We chose this application because it solves a real problem We personally face during exams—spending too much time planning instead of studying. Unlike traditional projects like library or hospital management systems, StudyCore is practical and directly useful. It also allowed us to apply Object-Oriented Programming concepts while creating something unique and meaningful.

➢ CONCLUSION

StudyCore is a simple yet powerful application that improves productivity by automating study planning. It saves time, and helps students focus on effective preparation. Overall, it demonstrates how programming can be used to solve real-life problems in an efficient way.

Task B — Design Pattern Identification

➢ Pattern 1:

Pattern Name and Category:

Strategy — Behavioral Pattern

Where and How It Appears:

The Strategy Patterns appears through three classes: Planner (abstract interface) , Time

Table Planner (concrete strategy) , and Study Context (contex class) . Study Context holds a planner pointer and calls generate plan through it polymorphically. It never knows which concrete strategy is plugged in. The strategy is injected at run time via context.setPlanner (&planner)

```
Class Planner {
 Public:
    Virtual void generatePlan(vector<Subject>&
Subjects)  = 0;
Virtual ~Planner() {}
};

Class TimeTablePlanner : public Planner{
Public:
  void generatePlan(vector<Subject>& Subjects)
override { }
  };

Class StudyContext{
 Private:
  Planner *strategy;
Public:
StudyContext() : strategy(NULL){}
Void setPlanner(Planner *s){ strategy = s ;}
Void executePlan(vector<Subject>& Subjects){
 If (strategy)
  Strategy -> generatePlan(subjects);
}
};
StudyFacade() {
  Context.setPlanner(&planner);
}
```

Why It Is Appropriate:

New planning strategies like CrashCoursePlanner can be added simply by subclassing Planner and overriding generatePlan(), with zero changes to StudyContext or StudyFacade. This keeps the system flexible and follows the Open / Closed Principle.

Problem It Solves:

It separates the scheduling algorithm from the rest of the system. The context is completely independent of which algorithm is running. This makes the system flexible and extensible for future planning strategies.

OOP Principles It Reinforces:

Abstraction through the Planner interface, Encapsulation by hiding the algorithm inside the concrete strategy class, and Polymorphism through the Planner pointer calling different implementations at runtime.

➢ Pattern 2

Pattern Name and Category:

Facade — Structural Pattern.

Where and How It Appears:

A Facade structure is implemented through the functions manageSubjects() and studyProgress(). These functions act as simplified interfaces hiding complex internal operations from main(). The main program interacts with complex subsystems through single clean function calls:

```
// main() — entire application runs through one simple call
int main() {
   StudyFacade app;
   app.start();
   return 0;
}

// Inside StudyFacade — all complexity hidden here
class StudyFacade {
private:
   vector<Subject>  subjects;
   TimetablePlanner planner;
   StudyContext     context;
   string           userName;
   string           userId;
public:
   void start()         { ... }
   void manageSubjects() { ... }
   void studyProgress()  { ... }
};
```

Why It Is Appropriate:

Without this structure all subject management and progress tracking logic would live inside main() making it extremely long and difficult to maintain. The Facade keeps main() clean and readable by hiding all internal complexity behind simple function calls.

Problem It Solves:

It reduces complexity at the top level of the program. Any developer reading main() immediately understands the entire program structure without needing to understand any internal details.

OOP Principles It Reinforces:

Abstraction and Encapsulation.

➢ Pattern 3 — Recommended Future Extension

Pattern Name and Category:

Iterator — Behavioral Pattern.

Where and How It Appears:

Currently subject traversal is performed using indexed for loops in generatePlan(), manageSubjects(), and studyProgress(). This tightly couples traversal logic with business logic. The Iterator pattern would introduce a dedicated class to handle traversal independently:

```
class SubjectIterator {
private:
   vector<Subject>& subjects;
   int index;
public:
   SubjectIterator(vector<Subject>& s) : subjects(s),
index(0) {}
   bool hasNext() { return index < subjects.size(); }
   Subject& next() { return subjects[index++]; }
};
```

Why It Is Appropriate:

It encapsulates traversal logic in one dedicated place allowing uniform access to subjects across all functions. This improves maintainability as the system grows and cleanly separates traversal responsibility from business logic.

Problem It Solves:

It removes tight coupling between traversal and business logic across generatePlan(), manageSubjects(), and studyProgress(). Centralizing traversal in a dedicated SubjectIterator class ensures data structure changes don't ripple through multiple functions.

OOP Principles It Reinforces:

Encapsulation and Abstraction.

Task C -- C++ Implementation

➢ Pattern 1 -- Behavioral Pattern (Strategy)

➢ Source Code:

```cpp
#include <iostream>
#include <vector>
#include <string>
using namespace std;


// ---------------------------------------------------------------------------
// Subject — data class
// Holds all information about one subject: name, chapters, exam days, weakness
// ---------------------------------------------------------------------------
class Subject {
private:
    string name;
    int    totalChapters;
    int    daysUntilExam;
    int    weaknessLevel;
    int    chaptersCompleted;

public:
    Subject(string n, int chapters, int days, int weakness)
        : name(n), totalChapters(chapters),
          daysUntilExam(days), weaknessLevel(weakness),
          chaptersCompleted(0) {}

    string getName()              const { return name; }
    int    getTotalChapters()     const { return totalChapters; }
    int    getDaysUntilExam()     const { return daysUntilExam; }
    int    getWeaknessLevel()     const { return weaknessLevel; }
    int    getChaptersCompleted() const { return chaptersCompleted; }
    int    getChaptersLeft()      const { return totalChapters - chaptersCompleted; }
};
```

```cpp
class TimetablePlanner : public Planner {
public:
    // Overrides Planner::generatePlan() — this is the concrete algorithm
    void generatePlan(vector<Subject>& subjects) override {

        if (subjects.empty()) {
            cout << "\n  No subjects added yet!\n";
            return;
        }

        cout << "\n  +==========================================+\n";
        cout << "  |       StudyCore -- Smart Timetable       |\n";
        cout << "  +==========================================+\n";

        // Sort by soonest exam first — bubble sort (from original project)
        int n = subjects.size();
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (subjects[j].getDaysUntilExam() > subjects[j+1].getDaysUntilExam()) {
                    Subject temp  = subjects[j];
                    subjects[j]   = subjects[j+1];
                    subjects[j+1] = temp;
                }
            }
        }
```

```cpp
for (int i = 0; i < (int)subjects.size(); i++) {

    int totalDays    = subjects[i].getDaysUntilExam();
    int chaptersLeft = subjects[i].getChaptersLeft();
    int done         = subjects[i].getChaptersCompleted();

    cout << "\n  ==========================================\n";
    cout << "  Subject : " << subjects[i].getName() << "\n";
    cout << "  ==========================================\n";
    cout << "  Chapters Left   : " << chaptersLeft << "\n";
    cout << "  Days Until Exam : " << totalDays    << "\n";
    cout << "  ------------------------------------------\n";

    // Case 1 — all chapters already done
    if (chaptersLeft <= 0) {
        cout << "  All chapters done! Revision plan:\n\n";
        for (int day = 1; day <= totalDays; day++)
            cout << "  Day " << day << " : Revision\n";
        continue;
    }

    // Case 2 — only 1 day left
    if (totalDays == 1) {
        cout << "  Only 1 day left!\n";
        cout << "  Day 1 : Revise everything + cover what you can\n";
        continue;
    }
```

```
        // Case 3 — normal smart calculation

        int availableStudyDays = totalDays - 1;
        int chaptersPerDay;
        int daysNeeded;
        int extraChapters;

        if (chaptersLeft <= availableStudyDays) {
        daysNeeded     = chaptersLeft;
        chaptersPerDay = 1;
        extraChapters  = 0;
        }

        else {
        daysNeeded     = availableStudyDays;
        chaptersPerDay = chaptersLeft / daysNeeded;
        extraChapters  = chaptersLeft % daysNeeded;
        }

int revisionDays = totalDays - daysNeeded;
```

```
    // Last day always revision
    if (day == totalDays) {
        cout << "  Day " << day << " : *** REVISION DAY ***\n";
        break;
    }

    // Past study days — revision
    if (chapterNumber > subjects[i].getTotalChapters() ||
        day > daysNeeded) {
        cout << "  Day " << day << " : Revision\n";
        continue;
    }

    int todayChapters = chaptersPerDay;
    if (extraChapters > 0 && day <= extraChapters)
        todayChapters = chaptersPerDay + 1;

    int chapterEnd = chapterNumber + todayChapters - 1;
    if (chapterEnd > subjects[i].getTotalChapters())
        chapterEnd = subjects[i].getTotalChapters();

    cout << "  Day " << day << " : Chapter "
         << chapterNumber << " to " << chapterEnd;

    // Weakness warning from original project
    if (subjects[i].getWeaknessLevel() >= 7)
        cout << "  [ Weak - Focus Hard! ]";

    cout << "\n";
    chapterNumber = chapterEnd + 1;
}

cout << "\n";
```

```
        if (chaptersPerDay >= 4)
            cout << "  Warning : Heavy load - " << chaptersPerDay << " chapters/day!\n";

        if (subjects[i].getWeaknessLevel() >= 8)
            cout << "  Tip : Weakest subject - start it first every day.\n";
        else if (subjects[i].getWeaknessLevel() >= 5)
            cout << "  Tip : Don't skip this subject.\n";
        else
            cout << "  Tip : You are strong here - stay consistent.\n";
    }

    cout << "\n  +==========================================+\n";
    cout << "  |   Timetable Ready! Stay Consistent!      |\n";
    cout << "  +==========================================+\n";
  }
};
```

```
// ---------------------------------------------------------------------------
// StudyContext — Context Class
//
// This is the CONTEXT in the Strategy Pattern. It holds a pointer to a
// Planner strategy and calls generatePlan() through it. It does not know
// or care which concrete strategy is plugged in — it just delegates.
// Calling setPlanner() at runtime swaps the algorithm with no other changes.
// ---------------------------------------------------------------------------
class StudyContext {
private:
    Planner* strategy;   // Pointer to current strategy — set at runtime

public:
    StudyContext() : strategy(NULL) {}

    // Plug in any Planner subclass — swaps algorithm at runtime
    void setPlanner(Planner* s) {
        strategy = s;
    }

    // Delegates execution to the current strategy via the Planner interface
    void executePlan(vector<Subject>& subjects) {
        if (strategy)
            strategy->generatePlan(subjects);   // Polymorphic call
        else
            cout << "\n  No planning strategy set!\n";
    }
};
```

```
// ==========================================================================
// main() — Demonstrates the Strategy Pattern
//
// - Creates subjects directly (no facade here — this file is Strategy only)
// - Creates a TimetablePlanner and plugs it into StudyContext via setPlanner()
// - Calls executePlan() — context delegates to the strategy polymorphically
// ==========================================================================
int main() {



    // Create subjects
    vector<Subject> subjects;
    subjects.push_back(Subject("Mathematics", 10, 7,  8));
    subjects.push_back(Subject("Physics",      6, 5,  5));
    subjects.push_back(Subject("English",      4, 10, 2));

    // Create the concrete strategy
    TimetablePlanner planner;

    // Create the context and plug in the strategy
    StudyContext context;
    context.setPlanner(&planner);   // Strategy set at runtime

    // Context delegates to TimetablePlanner via Planner* — Strategy Pattern
    context.executePlan(subjects);

    return 0;
}
```

➢ Sample Output:

```
+==========================================+
|       StudyCore -- Smart Timetable       |
+==========================================+

==========================================
Subject : eng
==========================================
Chapters Left   : 8
Days Until Exam : 6
------------------------------------------
Study Days    : 5
Revision Days : 1

Day 1 : Chapter 1 to 2  [ Weak - Focus Hard! ]
Day 2 : Chapter 3 to 4  [ Weak - Focus Hard! ]
Day 3 : Chapter 5 to 6  [ Weak - Focus Hard! ]
Day 4 : Chapter 7 to 7  [ Weak - Focus Hard! ]
Day 5 : Chapter 8 to 8  [ Weak - Focus Hard! ]
Day 6 : *** REVISION DAY ***

Tip : Don't skip this subject.

==========================================



Days Until Exam : 8
------------------------------------------
Study Days    : 7
Revision Days : 1

Day 1 : Chapter 1 to 1  [ Weak - Focus Hard! ]
Day 2 : Chapter 2 to 2  [ Weak - Focus Hard! ]
Day 3 : Chapter 3 to 3  [ Weak - Focus Hard! ]
Day 4 : Chapter 4 to 4  [ Weak - Focus Hard! ]
Day 5 : Chapter 5 to 5  [ Weak - Focus Hard! ]
Day 6 : Chapter 6 to 6  [ Weak - Focus Hard! ]
Day 7 : Chapter 7 to 7  [ Weak - Focus Hard! ]
Day 8 : *** REVISION DAY ***

Tip : Weakest subject - start it first every day.

+==========================================+
|   Timetable Ready! Stay Consistent!      |
+==========================================+
```

➢ Pattern 2 -- Structural Pattern (Facade)

➢ Source Code:

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;


// Subsystem A – stores data for one subject
class Subject {
private:
    string name;
    int totalChapters, daysUntilExam, weaknessLevel;

public:
    // Constructor to initialize subject data
    Subject(string n, int c, int d, int w)
        : name(n), totalChapters(c), daysUntilExam(d), weaknessLevel(w) {}

    // Getter functions (access private data)
    string getName()          const { return name; }
    int getTotalChapters()    const { return totalChapters; }
    int getDaysUntilExam()    const { return daysUntilExam; }
    int getWeaknessLevel()    const { return weaknessLevel; }

    // Setter functions (modify private data)
    void setName(string n)       { name = n; }
    void setTotalChapters(int c) { totalChapters = c; }
    void setDaysUntilExam(int d) { daysUntilExam = d; }
    void setWeaknessLevel(int w) { weaknessLevel = w; }
```

```
    // Display subject details
    void display(int index) const {
        cout << "\n  -- Subject " << index << " --\n";
        cout << "  Name            : " << name          << "\n";
        cout << "  Total Chapters  : " << totalChapters << "\n";
        cout << "  Days Until Exam : " << daysUntilExam << "\n";
        cout << "  Weakness (1-10) : " << weaknessLevel << "\n";
    }
};


// Subsystem B – handles add, edit, delete, view operations
class SubjectManager {
private:
    vector<Subject> subjects;  // Stores all subjects
```

```
public:
    // Add a new subject to the list
    void addSubject(string name, int chapters, int days, int weakness) {
        subjects.push_back(Subject(name, chapters, days, weakness));
        cout << "  Subject '" << name << "' added successfully.\n";
    }

    // Edit an existing subject by index
    void editSubject(int index, string name, int chapters, int days, int weakness) {
        int i = index - 1;  // Convert to 0-based index

        // Check if index is valid
        if (i < 0 || i >= (int)subjects.size()) {
            cout << "  Invalid subject number.\n";
            return;
        }

        // Update subject details
        subjects[i].setName(name);
        subjects[i].setTotalChapters(chapters);
        subjects[i].setDaysUntilExam(days);
        subjects[i].setWeaknessLevel(weakness);

        cout << "  Subject updated successfully.\n";
    }

    // Delete a subject by index
    void deleteSubject(int index) {
        int i = index - 1;  // Convert to 0-based index

        // Validate index
        if (i < 0 || i >= (int)subjects.size()) {
            cout << "  Invalid subject number.\n";
            return;
        }
```

```
        string deleted = subjects[i].getName();  // Store name before deleting
        subjects.erase(subjects.begin() + i);    // Remove subject

        cout << "  '" << deleted << "' deleted successfully.\n";
    }

    // Display all subjects
    void viewSubjects() {
        if (subjects.empty()) {
            cout << "  No subjects added yet.\n";
            return;
        }

        cout << "\n  +==========================================+\n";
        cout << "  |              ALL SUBJECTS                  |\n";
        cout << "  +==========================================+\n";

        // Loop through all subjects and display
        for (int i = 0; i < (int)subjects.size(); i++)
            subjects[i].display(i + 1);
    }
};
```

```cpp
// Facade — single entry point for all subject management
// Hides SubjectManager complexity from main()
class StudyFacade {
private:
    SubjectManager manager;  // Internal subsystem

public:
    // Forward calls to SubjectManager
    void addSubject(string name, int chapters, int days, int weakness) {
        manager.addSubject(name, chapters, days, weakness);
    }

    void editSubject(int index, string name, int chapters, int days, int weakness) {
        manager.editSubject(index, name, chapters, days, weakness);
    }

    void deleteSubject(int index) {
        manager.deleteSubject(index);
    }

    void viewSubjects() {
        manager.viewSubjects();
    }
};
```

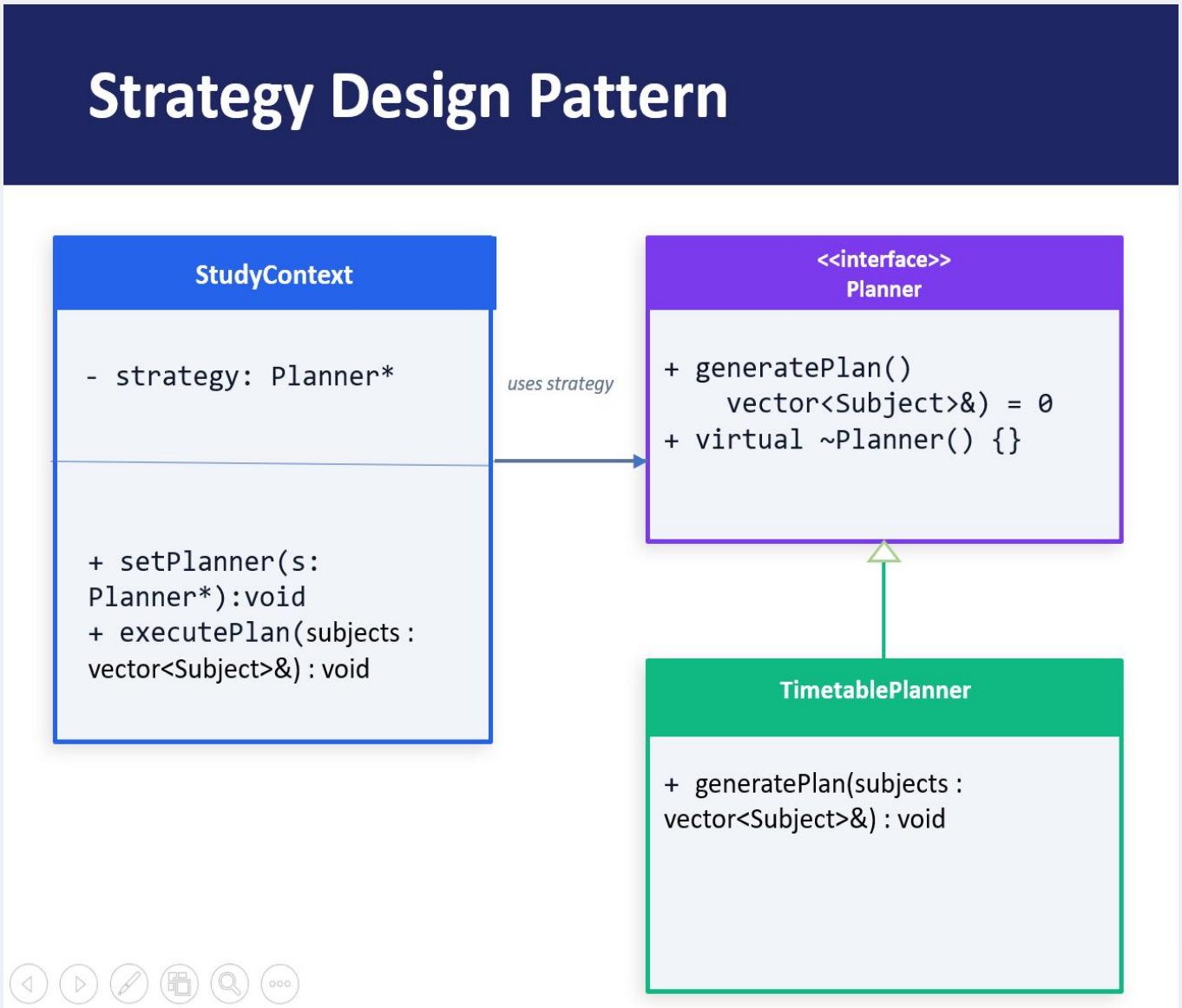
```cpp
// main() — demonstrates Facade Pattern
// Only interacts with StudyFacade (not internal classes)
int main() {

    StudyFacade app;  // Single interface object

    // Add subjects
    cout << "\n  -- Adding Subjects --\n";
    app.addSubject("Mathematics", 10, 7,  8);
    app.addSubject("English",      4, 10, 2);

    // View subjects
    cout << "\n  -- Viewing All Subjects --\n";
    app.viewSubjects();

    // Edit subject
    cout << "\n  -- Editing Subject 2 (Physics) --\n";
    app.editSubject(2, "Physics", 8, 6, 6);

    // Delete subject
    cout << "\n  -- Deleting Subject 3 (English) --\n";
    app.deleteSubject(3);

    // View after deleting and editing
    cout << "\n  -- Viewing After Deleting and Editing --\n";
    app.viewSubjects();

    return 0;
}
```

➢ Sample Output:

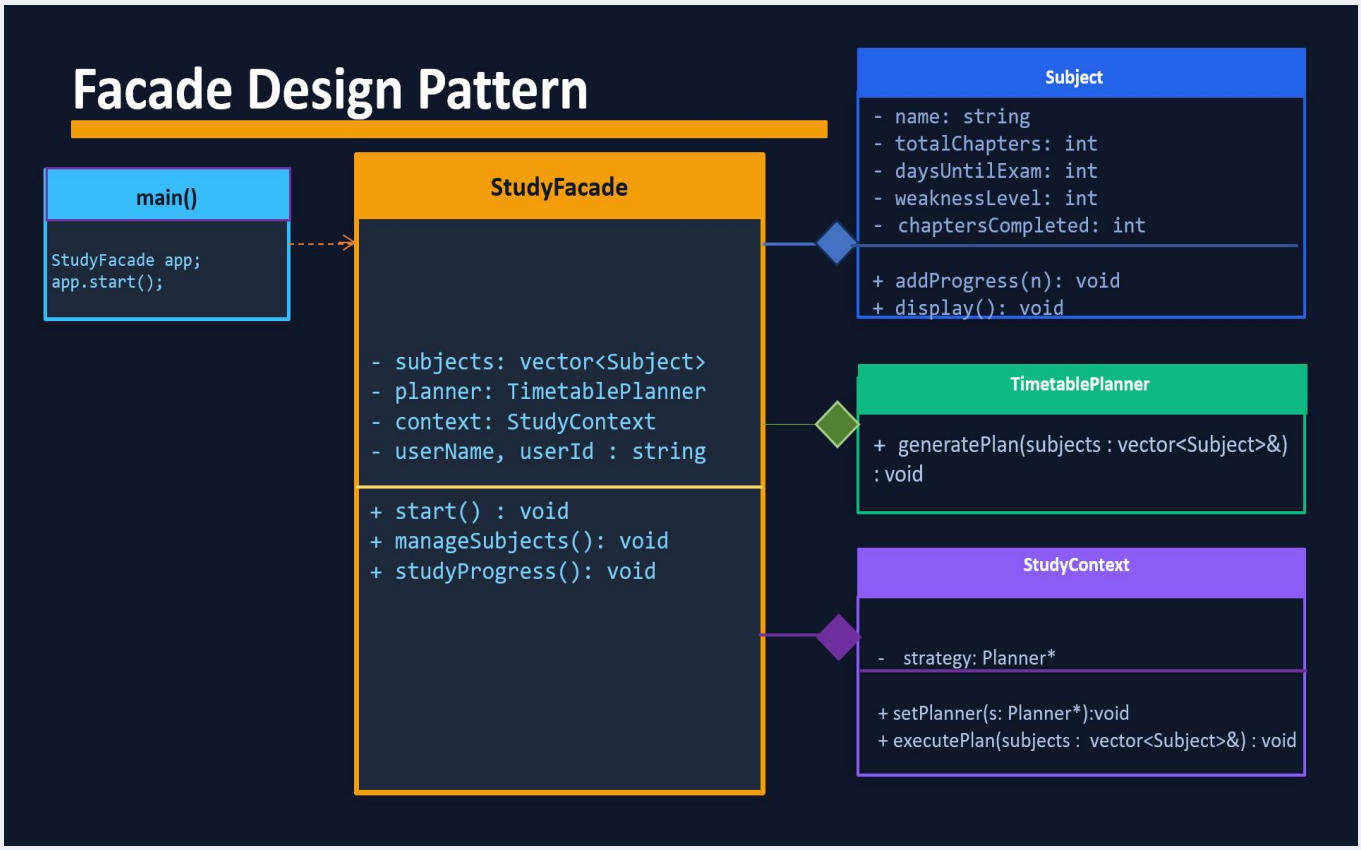
```
-- Adding Subjects --
Subject 'Mathematics' added successfully.
Subject 'English' added successfully.

-- Viewing All Subjects --

+==========================================+
|            ALL SUBJECTS                  |
+==========================================+

-- Subject 1 --
Name            : Mathematics
Total Chapters  : 10
Days Until Exam : 7
Weakness (1-10) : 8

-- Subject 2 --
Name            : English
Total Chapters  : 4
Days Until Exam : 10
Weakness (1-10) : 2

-- Editing Subject 2 (Physics) --
Subject updated successfully.

-- Deleting Subject 3 (English) --
Invalid subject number.

-- Viewing After Deleting and Editing --

+==========================================+
|            ALL SUBJECTS                  |
+==========================================+

-- Subject 1 --
Name            : Mathematics
Total Chapters  : 10
Days Until Exam : 7
Weakness (1-10) : 8

-- Subject 2 --
Name            : Physics
Total Chapters  : 8
Days Until Exam : 6
Weakness (1-10) : 6
```

Task D -- UML Diagrams:

- **UML Diagram 1:**



➢ UML Diagram 2:



Appendix — Complete Source Code of StudyCore:

The following images contain the complete C++ source code of the StudyCore application, demonstrating the integration of the Strategy and Facade design patterns:

```
#include <iostream>

#include <vector>

#include <string>

using namespace std;



//
==========================================================
=================

// CLASS 1: Subject

// Represents one subject with all its data

//
==========================================================
=================

class Subject {

private:

    string name;

    int totalChapters;

    int daysUntilExam;

    int weaknessLevel;

    int chaptersCompleted;
```

```
public:
    Subject(string n, int chapters, int days, int weakness) {
        name = n;
        totalChapters = chapters;
        daysUntilExam = days;
        weaknessLevel = weakness;
        chaptersCompleted = 0;
    }

    // Getters
    string getName()          { return name; }
    int getTotalChapters()    { return totalChapters; }
    int getDaysUntilExam()    { return daysUntilExam; }
    int getWeaknessLevel()    { return weaknessLevel; }
    int getChaptersCompleted() { return chaptersCompleted; }
    int getChaptersLeft()     { return totalChapters - chaptersCompleted; }

    // Setters
    void setName(string n)      { name = n; }
    void setTotalChapters(int c) { totalChapters = c; }
    void setDaysUntilExam(int d) { daysUntilExam = d; }
    void setWeaknessLevel(int w) { weaknessLevel = w; }
```

```
void addProgress(int count) {
    if (getChaptersLeft() == 0) {
        cout << "\n  All chapters already completed!\n";
        return;
    }
    int remaining = totalChapters - chaptersCompleted;
    while (count < 1 || count > remaining) {
        if (count < 1)
            cout << "  Enter at least 1 chapter.\n";
        else
            cout << "  Only " << remaining << " chapters left! Enter correct number:
";
        cin >> count;
        cin.ignore();
    }
    chaptersCompleted += count;
}

void display() {
    cout << "  Subject        : " << name << "\n";
    cout << "  Total Chapters  : " << totalChapters << "\n";
    cout << "  Chapters Done   : " << chaptersCompleted << "\n";
```

```
        cout << "  Chapters Left   : " << getChaptersLeft() << "\n";
        cout << "  Days Until Exam : " << daysUntilExam << "\n";
        cout << "  Weakness (1-10) : " << weaknessLevel << "\n";
    }
};


//
==================================================================
// CLASS 2: Planner — Strategy Interface (Abstract Base Class)
// Defines the contract — every planner MUST implement generatePlan()
// Pure virtual function forces all child classes to override it
//
==================================================================
class Planner {
public:
    virtual void generatePlan(vector<Subject>& subjects) = 0;
    virtual ~Planner() {}
};


//
==================================================================
// CLASS 3: TimetablePlanner — Concrete Strategy
```

```
// Inherits from Planner and implements the actual timetable algorithm
//
==========================================================
=================
class TimetablePlanner : public Planner {
public:
   void generatePlan(vector<Subject>& subjects) override {


      if (subjects.empty()) {
         cout << "\n  No subjects added yet!\n";
         cout << "  Go to Manage Subjects and add subjects first.\n";
         return;
      }


      cout << "\n
+=========================================+\n";
      cout << "  |      StudyCore -- Smart Timetable      |\n";
      cout << "
+=========================================+\n";


      // Sort subjects by soonest exam first — bubble sort
      int n = subjects.size();
      for (int i = 0; i < n - 1; i++) {
         for (int j = 0; j < n - i - 1; j++) {
```

```
            if (subjects[j].getDaysUntilExam() > subjects[j+1].getDaysUntilExam())
{
                Subject temp  = subjects[j];
                subjects[j]   = subjects[j+1];
                subjects[j+1] = temp;
            }
        }
    }

    for (int i = 0; i < subjects.size(); i++) {

        int totalDays    = subjects[i].getDaysUntilExam();
        int chaptersLeft = subjects[i].getChaptersLeft();
        int done         = subjects[i].getChaptersCompleted();

        cout << "\n
==========================================\n";
        cout << "  Subject : " << subjects[i].getName() << "\n";
        cout << "  ==========================================\n";
        cout << "  Chapters Left   : " << chaptersLeft << "\n";
        cout << "  Days Until Exam : " << totalDays << "\n";
        cout << "  ------------------------------------------\n";
```

```
// Case 1 — all chapters done
if (chaptersLeft <= 0) {
    cout << "  All chapters done! Revision plan:\n\n";
    for (int day = 1; day <= totalDays; day++)
        cout << "  Day " << day << " : Revision\n";
    continue;
}

// Case 2 — only 1 day left
if (totalDays == 1) {
    cout << "  Only 1 day left!\n";
    cout << "  Day 1 : Revise everything + cover what you can\n";
    continue;
}

// Case 3 — normal smart calculation
// minimum 1 chapter per day
// only use days needed — rest become revision
int chaptersPerDay = 1;
int daysNeeded     = chaptersLeft;

if (daysNeeded >= totalDays) {
    daysNeeded     = totalDays - 1;
```

```
    chaptersPerDay = (chaptersLeft + daysNeeded - 1) / daysNeeded;
}

if (chaptersPerDay == 0)
    chaptersPerDay = 1;

int extraChapters = chaptersLeft % daysNeeded;
int revisionDays  = totalDays - daysNeeded;

cout << "  Study Days    : " << daysNeeded << "\n";
cout << "  Revision Days : " << revisionDays << "\n\n";

int chapterNumber = done + 1;

for (int day = 1; day <= totalDays; day++) {

    // last day always revision
    if (day == totalDays) {
        cout << "  Day " << day << " : *** REVISION DAY ***\n";
        break;
    }

    // chapters finished or past study days — revision
```

```
if (chapterNumber > subjects[i].getTotalChapters() ||
    day > daysNeeded) {
    cout << "  Day " << day << " : Revision\n";
    continue;
}

int todayChapters = chaptersPerDay;
if (extraChapters > 0 && day <= extraChapters)
    todayChapters = chaptersPerDay + 1;

int chapterEnd = chapterNumber + todayChapters - 1;

if (chapterEnd > subjects[i].getTotalChapters())
    chapterEnd = subjects[i].getTotalChapters();

cout << "  Day " << day << " : Chapter "
     << chapterNumber << " to " << chapterEnd;

if (subjects[i].getWeaknessLevel() >= 7)
    cout << "  [ Weak - Focus Hard! ]";

cout << "\n";
chapterNumber = chapterEnd + 1;
```

```
        }

        cout << "\n";

        if (chaptersPerDay >= 4)
            cout << "  Warning : Heavy load - " << chaptersPerDay
                << " chapters/day!\n";

        if (subjects[i].getWeaknessLevel() >= 8)
            cout << "  Tip : Weakest subject - Focus more on it.\n";
        else if (subjects[i].getWeaknessLevel() >= 5)
            cout << "  Tip : Don't skip this subject.\n";
        else
            cout << "  Tip : You are strong here - stay consistent.\n";
    }

    cout << "\n
+==========================================+\n";
    cout << "  |   Timetable Ready! Stay Consistent!      |\n";
    cout << "
+==========================================+\n";
  }
};
```

```
//
==========================================================
=================
// CLASS 4: StudyContext — Strategy Context
// Holds a pointer to a Planner strategy
// Allows switching planning algorithms without changing other code
//
==========================================================
=================
class StudyContext {
private:
    Planner* strategy;
public:
    StudyContext() : strategy(NULL) {}

    void setPlanner(Planner* s) {
        strategy = s;
    }

    void executePlan(vector<Subject>& subjects) {
        if (strategy)
            strategy->generatePlan(subjects);
    }
```

```
};


//
==========================================================
=================

// CLASS 5: StudyFacade — Facade Pattern

// Hides all complexity of subject management, progress tracking,

// and timetable generation behind one simple class

// main() only creates this object and calls start()

//
==========================================================
=================

class StudyFacade {

private:

    vector<Subject>  subjects;

    TimetablePlanner planner;

    StudyContext     context;

    string           userName;

    string           userId;


public:

    StudyFacade() {

        context.setPlanner(&planner);

    }
```

```
void start() {
    cout << "\n
+==========================================+\n";
    cout << "  |        WELCOME TO STUDYCORE           |\n";
    cout << "  |    Intelligent Study Engine v1.0        |\n";
    cout << "
+==========================================+\n";
    cout << "\n  Enter Your Name : ";
    getline(cin, userName);
    cout << "  Enter Your ID   : ";
    getline(cin, userId);
    cout << "\n  Welcome, " << userName << "! Profile created.\n";

    int choice;
    do {
        cout << "\n
+==========================================+\n";
        cout << "  |             DASHBOARD                 |\n";
        cout << "
+==========================================+\n";
        cout << "  1. View Profile\n";
        cout << "  2. Manage Subjects\n";
        cout << "  3. Generate Timetable\n";
```

```
        cout << "  4. Study Progress\n";
        cout << "  0. Exit\n";
        cout << "\n  Enter Choice: ";
        cin >> choice;
        cin.ignore();

        if (choice == 1) {
            cout << "\n
+=========================================+\n";
            cout << "  |          STUDENT PROFILE            |\n";
            cout << "
+=========================================+\n";
            cout << "  Name : " << userName << "\n";
            cout << "  ID   : " << userId   << "\n";
        }
        else if (choice == 2)
            manageSubjects();
        else if (choice == 3)
            context.executePlan(subjects);
        else if (choice == 4)
            studyProgress();
        else if (choice == 0) {
            cout << "\n
+=========================================+\n";
```

```
            cout << "  |  Goodbye, " << userName << "!\n";
            cout << "  |  Keep Studying. You Got This!\n";
            cout << "
+==========================================+\n\n";
        }
        else
            cout << "\n  Invalid choice. Please try again.\n";

    } while (choice != 0);
  }

  void manageSubjects() {
    int choice;
    do {
        cout << "\n
+==========================================+\n";
        cout << "  |          MANAGE SUBJECTS              |\n";
        cout << "
+==========================================+\n";
        cout << "  1. Add Subject\n";
        cout << "  2. Edit Subject\n";
        cout << "  3. Delete Subject\n";
        cout << "  4. View Subjects\n";
        cout << "  0. Back to Dashboard\n";
```

```
cout << "\n  Enter Choice: ";
cin >> choice;
cin.ignore();

if (choice == 1) {
    string n;
    int c, d, w;

    cout << "\n  -- Add New Subject --\n";
    cout << "  Subject Name           : ";
    getline(cin, n);

    do {
        cout << "  Total Chapters (1-100)  : ";
        cin >> c;
        if (c < 1 || c > 200)
            cout << "  Invalid! Enter between 1 and 100.\n";
    } while (c < 1 || c > 200);

    do {
        cout << "  Days Until Exam (1-365) : ";
        cin >> d;
        if (d < 1 || d > 365)
```

```
            cout << "  Invalid! Enter between 1 and 365.\n";
        } while (d < 1 || d > 365);

        do {
            cout << "  Weakness Level (1-10)   : ";
            cin >> w;
            if (w < 1 || w > 10)
                cout << "  Invalid! Enter between 1 and 10.\n";
        } while (w < 1 || w > 10);

        cin.ignore();
        subjects.push_back(Subject(n, c, d, w));
        cout << "\n  Subject added successfully!\n";
    }

    else if (choice == 2) {
        if (subjects.empty()) {
            cout << "\n  No subjects to edit.\n";
        } else {
            cout << "\n  -- Select Subject to Edit --\n";
            for (int i = 0; i < subjects.size(); i++)
                cout << "  [" << i+1 << "] " << subjects[i].getName() << "\n";
```

```
int idx;
do {
    cout << "  Enter number: ";
    cin >> idx;
    if (idx < 1 || idx > subjects.size())
        cout << "  Invalid! Enter between 1 and "
            << subjects.size() << ".\n";
} while (idx < 1 || idx > subjects.size());

cin.ignore();
idx = idx - 1;

string n;
int c, d, w;

cout << "  New  Subject Name              : ";
getline(cin, n);

do {
    cout << "  New Total Chapters (1-200)  : ";
    cin >> c;
    if (c < 1 || c > 200)
        cout << "  Invalid! Enter between 1 and 200.\n";
```

```
    } while (c < 1 || c > 200);

    do {
        cout << "  New Days Until Exam (1-365) : ";
        cin >> d;
        if (d < 1 || d > 365)
            cout << "  Invalid! Enter between 1 and 365.\n";
    } while (d < 1 || d > 365);

    do {
        cout << "  New Weakness (1-10)        : ";
        cin >> w;
        if (w < 1 || w > 10)
            cout << "  Invalid! Enter between 1 and 10.\n";
    } while (w < 1 || w > 10);

    cin.ignore();
    subjects[idx].setName(n);
    subjects[idx].setTotalChapters(c);
    subjects[idx].setDaysUntilExam(d);
    subjects[idx].setWeaknessLevel(w);
    cout << "\n  Subject updated successfully!\n";
}
```

```
    }

    else if (choice == 3) {
        if (subjects.empty()) {
            cout << "\n  No subjects to delete.\n";
        } else {
            cout << "\n  -- Select Subject to Delete --\n";
            for (int i = 0; i < subjects.size(); i++)
                cout << "  [" << i+1 << "] " << subjects[i].getName() << "\n";

            int idx;
            do {
                cout << "  Enter number: ";
                cin >> idx;
                if (idx < 1 || idx > subjects.size())
                    cout << "  Invalid! Enter between 1 and "
                         << subjects.size() << ".\n";
            } while (idx < 1 || idx > subjects.size());

            cin.ignore();
            idx = idx - 1;
            string deleted = subjects[idx].getName();
            subjects.erase(subjects.begin() + idx);
```

```
            cout << "\n  '" << deleted << "' deleted successfully.\n";
        }
    }

    else if (choice == 4) {
        if (subjects.empty()) {
            cout << "\n  No subjects added yet.\n";
        } else {
            cout << "\n
+=========================================+\n";
            cout << "  |          ALL SUBJECTS              |\n";
            cout << "
+=========================================+\n";
            for (int i = 0; i < subjects.size(); i++) {
                cout << "\n  -- Subject " << i+1 << " --\n";
                subjects[i].display();
            }
        }
    }

    else if (choice != 0)
        cout << "\n  Invalid choice. Please try again.\n";
```

```
    } while (choice != 0);
  }


  void studyProgress() {
    int choice;
    do {
      cout << "\n
+=========================================+\n";
      cout << "  |          STUDY PROGRESS            |\n";
      cout << "
+=========================================+\n";
      cout << "  1. Mark Progress\n";
      cout << "  2. View Progress\n";
      cout << "  0. Back to Dashboard\n";
      cout << "\n  Enter Choice: ";
      cin >> choice;
      cin.ignore();


      if (choice == 1) {
        if (subjects.empty()) {
          cout << "\n  No subjects added yet.\n";
        } else {
          cout << "\n  -- Select Subject --\n";
```

```
for (int i = 0; i < subjects.size(); i++)
    cout << "  [" << i+1 << "] " << subjects[i].getName() << "\n";

int idx;
do {
    cout << "  Enter number : ";
    cin >> idx;
    if (idx < 1 || idx > subjects.size())
        cout << "  Invalid! Enter between 1 and "
            << subjects.size() << ".\n";
} while (idx < 1 || idx > subjects.size());

cin.ignore();
idx = idx - 1;

cout << "  Chapters done today : ";
int count;
cin >> count;
cin.ignore();

subjects[idx].addProgress(count);
cout << "\n  Progress saved!\n";
cout << "  Chapters Left : "
```

```
                << subjects[idx].getChaptersLeft() << "\n";
            }
        }


        else if (choice == 2) {
            if (subjects.empty()) {
                cout << "\n  No subjects added yet.\n";
            } else {
                cout << "\n
+=========================================+\n";
                cout << "  |          STUDY PROGRESS              |\n";
                cout << "
+=========================================+\n";
                for (int i = 0; i < subjects.size(); i++) {
                    cout << "\n  -- " << subjects[i].getName() << " --\n";
                    cout << "  Chapters Done : "
                        << subjects[i].getChaptersCompleted() << "\n";
                    cout << "  Chapters Left : "
                        << subjects[i].getChaptersLeft() << "\n";
                    cout << "  Total        : "
                        << subjects[i].getTotalChapters() << "\n";
                }
            }
```

```
        }

        else if (choice != 0)
            cout << "\n  Invalid choice. Please try again.\n";

    } while (choice != 0);
  }
};



int main() {
  StudyFacade app;
  app.start();
  return 0;
}
```

➢ Sample Output:

```
+==========================================+
|         WELCOME TO STUDYCORE             |
|    Intelligent Study Engine v1.0         |
+==========================================+

Enter Your Name : hafsa nasir
Enter Your ID   : 001

Welcome, hafsa nasir! Profile created.

+==========================================+
|              DASHBOARD                   |
+==========================================+
1. View Profile
2. Manage Subjects
3. Generate Timetable
4. Study Progress
0. Exit

Enter Choice: 1

+==========================================+
|            STUDENT PROFILE               |
+==========================================+
Name : hafsa nasir
ID   : 001

+==========================================+
|              DASHBOARD                   |
+==========================================+
1. View Profile
2. Manage Subjects
3. Generate Timetable
4. Study Progress
0. Exit

Enter Choice: 2

+==========================================+
|           MANAGE SUBJECTS                |
+==========================================+
1. Add Subject
2. Edit Subject
3. Delete Subject
4. View Subjects
0. Back to Dashboard

Enter Choice: 1
```

```
-- Add New Subject --
Subject Name            : eng
Total Chapters (1-200)  : 8
Days Until Exam (1-365) : 6
Weakness Level (1-10)   : 7

Subject added successfully!

+==========================================+
|           MANAGE SUBJECTS                |
+==========================================+
1. Add Subject
2. Edit Subject
3. Delete Subject
4. View Subjects
0. Back to Dashboard

Enter Choice: 1

-- Add New Subject --
Subject Name            : maths
Total Chapters (1-200)  : 7
Days Until Exam (1-365) : 8
Weakness Level (1-10)   : 10

Subject added successfully!

+==========================================+
|           MANAGE SUBJECTS                |
+==========================================+
1. Add Subject
2. Edit Subject
3. Delete Subject
4. View Subjects
0. Back to Dashboard

Enter Choice: 1

-- Add New Subject --
Subject Name            : urdu
Total Chapters (1-200)  : 7
Days Until Exam (1-365) : 8
Weakness Level (1-10)   : 4

Subject added successfully!

+==========================================+
|           MANAGE SUBJECTS                |
+==========================================+
```

```
1. Add Subject
2. Edit Subject
3. Delete Subject
4. View Subjects
0. Back to Dashboard

Enter Choice: 2

-- Select Subject to Edit --
[1] eng
[2] maths
[3] urdu
Enter number: 3
New Name                : isl
New Total Chapters (1-200)  : 7
New Days Until Exam (1-365) : 5
New Weakness (1-10)         : 8

Subject updated successfully!

+==========================================+
|           MANAGE SUBJECTS                |
+==========================================+
1. Add Subject
2. Edit Subject
3. Delete Subject
4. View Subjects
0. Back to Dashboard

Enter Choice: 3

-- Select Subject to Delete --
[1] eng
[2] maths
[3] isl
Enter number: 3

'isl' deleted successfully.

+==========================================+
|           MANAGE SUBJECTS                |
+==========================================+
1. Add Subject
2. Edit Subject
3. Delete Subject
4. View Subjects
0. Back to Dashboard

Enter Choice: 4
```

```
+==========================================+
|            ALL SUBJECTS                  |
+==========================================+

-- Subject 1 --
Subject         : eng
Total Chapters  : 8
Chapters Done   : 0
Chapters Left   : 8
Days Until Exam : 6
Weakness (1-10) : 7

-- Subject 2 --
Subject         : maths
Total Chapters  : 7
Chapters Done   : 0
Chapters Left   : 7
Days Until Exam : 8
Weakness (1-10) : 10

+==========================================+
|           MANAGE SUBJECTS                |
+==========================================+
1. Add Subject
2. Edit Subject
3. Delete Subject
4. View Subjects
0. Back to Dashboard

Enter Choice: 0

+==========================================+
|              DASHBOARD                   |
+==========================================+
1. View Profile
2. Manage Subjects
3. Generate Timetable
4. Study Progress
0. Exit

Enter Choice: 3

+==========================================+
|       StudyCore -- Smart Timetable       |
+==========================================+

==========================================
Subject : eng
==========================================
```

```
=========================================
Chapters Left   : 8
Days Until Exam : 6
-----------------------------------------
Study Days    : 5
Revision Days : 1

Day 1 : Chapter 1 to 2  [ Weak - Focus Hard! ]
Day 2 : Chapter 3 to 4  [ Weak - Focus Hard! ]
Day 3 : Chapter 5 to 6  [ Weak - Focus Hard! ]
Day 4 : Chapter 7 to 7  [ Weak - Focus Hard! ]
Day 5 : Chapter 8 to 8  [ Weak - Focus Hard! ]
Day 6 : *** REVISION DAY ***

Tip : Don't skip this subject.

=========================================
Subject : maths
=========================================
Chapters Left   : 7
Days Until Exam : 8
-----------------------------------------
Study Days    : 7
Revision Days : 1

Day 1 : Chapter 1 to 1  [ Weak - Focus Hard! ]
Day 2 : Chapter 2 to 2  [ Weak - Focus Hard! ]
Day 3 : Chapter 3 to 3  [ Weak - Focus Hard! ]
Day 4 : Chapter 4 to 4  [ Weak - Focus Hard! ]
Day 5 : Chapter 5 to 5  [ Weak - Focus Hard! ]
Day 6 : Chapter 6 to 6  [ Weak - Focus Hard! ]
Day 7 : Chapter 7 to 7  [ Weak - Focus Hard! ]
Day 8 : *** REVISION DAY ***

Tip : Weakest subject - start it first every day.

+=========================================+
|   Timetable Ready! Stay Consistent!     |
+=========================================+

+=========================================+
|              DASHBOARD                  |
+=========================================+
1. View Profile
2. Manage Subjects
3. Generate Timetable
4. Study Progress
0. Exit

Enter Choice: 4
```

```
+==========================================+
|            STUDY PROGRESS                |
+==========================================+
1. Mark Progress
2. View Progress
0. Back to Dashboard

Enter Choice: 1

-- Select Subject --
[1] eng
[2] maths
Enter number : 2
Chapters done today : 3

Progress saved!
Chapters Left : 4

+==========================================+
|            STUDY PROGRESS                |
+==========================================+
1. Mark Progress
2. View Progress
0. Back to Dashboard

Enter Choice: 2

+==========================================+
|            STUDY PROGRESS                |
+==========================================+

-- eng --
Chapters Done : 0
Chapters Left : 8
Total         : 8

-- maths --
Chapters Done : 3
Chapters Left : 4
Total         : 7

+==========================================+
|            STUDY PROGRESS                |
+==========================================+
1. Mark Progress
2. View Progress
0. Back to Dashboard

Enter Choice: 0
```

```
+=========================================+
|              DASHBOARD                  |
+=========================================+
1. View Profile
2. Manage Subjects
3. Generate Timetable
4. Study Progress
0. Exit

Enter Choice: 0

+=========================================+
|  Goodbye, hafsa nasir!
|  Keep Studying. You Got This!
+=========================================+
```